MDPI

Article

Time-Decayed Caching: A Novel Rule Benchmarked Against Established Policies

Cătălin Pângăleanu [1]

[1] Babeş-Bolyai University, Faculty of Mathematics and Computer Science, Cluj-Napoca, Romania; catalin.pangaleanu@stud.ubbcluj.ro

Abstract

This paper introduces Time-Decayed Caching (TDC), a cache replacement policy that models object popularity using an exponential half-life. Each access contributes one unit of "popularity mass" that decays over subsequent requests, controlled by a single half-life parameter . This yields a scalar score that simultaneously captures recency and frequency: recent hits dominate, while repeated hits accumulate mass over time. TDC is evaluated against three established policies, Least Recently Used (LRU), Least Frequently Used (LFU), and Adaptive Replacement Cache (ARC), on four synthetic traces and one real-world NASA HTTP log. On stationary workloads, TDC consistently improves on LRU and remains competitive with LFU and ARC. On non-stationary workloads with items of shifting popularity, TDC substantially outperforms LFU and preserves most of ARC's advantages, achieving fast recovery of its hit ratio after each phase change. On the real-world trace, TDC again improves over LRU and stays within a few percentage points of LFU and ARC across cache sizes. Overall, the results show that the time-decayed formulation can deliver robust, competitive performance while providing an explicit, interpretable control of the cache's memory horizon.

Keywords: cache replacement; caching; eviction policy; time-decay; recency; frequency; non-stationary workloads

MSC: 68M20, 68P05

1. Introduction

We propose a time-decayed policy based on the nuclear physics concept of half-life [1]. We introduce a one-parameter eviction rule that maintains a per-key decayed score using a half-life (H) constant. The policy blends recency and frequency and evicts the item with the lowest current decayed score.

1.1. Motivation

Caching remains one of the most effective tools for hiding latency and reducing load in modern systems, yet the choice of replacement policy is still highly workload-dependent. Classical rules such as LRU [2] and LFU [3] embody two extreme views of locality: LRU trusts recency alone, while LFU trusts long-term frequency. In practice, real workloads rarely conform to either extreme. They have bursty access patterns and are often non-stationary, with "hot" items drifting over time as user interests or application phases change. In such settings, purely recency-based policies adapt quickly but ignore valuable

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Algorithms* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

frequency information, whereas purely frequency-based policies achieve high hit ratios on stable traces but react too slowly when the hot set shifts.

A second limitation of many high-performing policies is the lack of a simple, explicit notion of how long past evidence should matter. Adaptive algorithms such as ARC [5] implicitly balance recency and frequency through complex list management and ghost structures, but they do not expose a tunable parameter that directly encodes the time horizon of relevance. This makes it difficult to reason mathematically about their behavior. There is therefore a gap for a replacement policy that

- unifies recency and frequency in a single, interpretable quantity
- allows the influence of past accesses to decay in a principled, time-controlled way, and
- remains simple enough to implement efficiently with standard data structures.

1.2. Contributions

This work introduces Time-Decayed Caching (TDC), a replacement policy built around an exponential half-life model of popularity. Our main contributions are:

1. **Half-life based formulation of popularity**

   We define, for each object, a time-decayed popularity score that aggregates all past hits with exponential decay controlled by a half-life parameter H. This formulation provides a single scalar that simultaneously captures recency (recent hits contribute more) and frequency (repeated hits accumulate mass), and it offers a clear interpretation: H is the horizon at which an old hit is worth half a new one.

2. **Efficient $O(\log C)$ implementation via time-normalized keys**

   We derive an algebraic transformation from the raw decayed scores to time-normalized keys K_i that preserve the ordering of objects by popularity. This allows the policy to store only K_i in a binary min-heap, achieving exact time-decayed eviction decisions with $O(\log C)$ worst-case work per request and $O(C)$ space, while avoiding full cache scans or per-request global decay.

3. **Practical design and open-source implementation**

   We specify the complete data structures and core operations required to implement TDC, and we provide a reference implementation suitable for integration into existing systems.

4. **Empirical evaluation against strong baselines**

   Using four synthetic traces (two stationary Zipf [3] workloads and two non-stationary phase-based workloads) and one real-world trace, we benchmark TDC against LRU, LFU, and ARC over a range of cache sizes. The experiments show that TDC consistently improves over LRU, substantially outperforms LFU on non-stationary workloads, and remains close to ARC across all workloads. TDC also exhibits smooth recovery of its hit ratio after the set of currently popular items (hot-set) changes.

1.3. Research Questions

- On stationary workloads (where the popular items requested from the cache never change), does time-decayed caching improve hit ratio over standard policies, such as Least Recently Used (LRU), Least Frequently Used (LFU), and Adaptive Replacement Cache (ARC)?
- On non-stationary workloads (where the distribution of popular items shifts periodically) does time-decayed caching improve hit ratio over standard policies?

https://doi.org/10.3390/a1010000

- Does time-decayed caching prove adaptivity by quickly recovering its hit ratio after shifts in the distribution of popular items occur in non-stationary workloads?

2. Background and Related Work

2.1. Background

Caching is a standard mechanism for reducing latency in data-intensive systems. A cache stores a subset of objects from a larger universe and serves future requests directly from this faster memory tier whenever possible. Each request is either a hit (the requested object is already in the cache) or a miss (the object must be fetched from the backing store and possibly inserted into the cache). Over a finite request trace, the primary effectiveness metric is the hit ratio, defined as the fraction of requests that are served as hits, with the complementary miss ratio measuring the fraction of requests that incur a miss.

When the cache is full, a replacement policy decides which object to evict to make room for a newly requested one. Classical policies such as Least Recently Used (LRU) and Least Frequently Used (LFU) exploit different forms of locality, and more sophisticated adaptive schemes (e.g. ARC) attempt to balance recency and frequency automatically. In this work, we adopt the standard setting of fixed-capacity object caches and regard a replacement policy as successful if, for a wide range of workloads and cache sizes, it achieves high hit ratios and small gaps to strong baselines such as LRU, LFU, and ARC, while remaining implementable with modest time and space overhead.

2.2. Existing Policies

We review the established caching policies selected as baselines for evaluation: Least Recently Used (LRU [2]), Least Frequently Used (LFU [3]), and Adaptive Replacement Cache (ARC [5]). We analyze their mechanisms in relation to our proposed Time-Decayed Caching (TDC) policy, highlighting key differences, similarities, and the specific scenarios where our approach is expected to outperform them.

Least Recently Used (LRU)

LRU is one of the most ubiquitous caching algorithms due to its simplicity and low overhead. It operates on the heuristic that items accessed recently are likely to be accessed again in the near future (temporal locality). The policy maintains a list of items ordered by access time; upon a cache miss, the item at the tail of the list (the least recently accessed) is evicted.

Comparison with Proposed Policy

- Similarities: Both LRU and TDC account for recency. In TDC, a recent access resets or significantly boosts an item's score, ensuring that recently requested items are unlikely to be evicted immediately.
- Differences: The fundamental difference lies in history retention. LRU is "memoryless" regarding frequency; it does not distinguish between an item accessed once and an item accessed a thousand times if both were last accessed at the same moment. In contrast, TDC integrates frequency by accumulating score mass from multiple accesses.
- Expected Performance: We expect TDC to outperform LRU on most workloads, especially ones where frequency is a strong predictor of future access. While LRU excels at adapting to sudden changes, it often suffers from "scan resistance" issues where a one-time scan of items flushes valuable popular content. TDC's accumulated score mass provides resistance against such scenarios.

Least Frequently Used (LFU)

LFU operates on the premise that items with a high frequency of access in the past are likely to be accessed frequently in the future. It maintains a counter for every item,

incrementing it upon every hit. When the cache is full, the item with the lowest frequency count is evicted.

Comparison with Proposed Policy

- Similarities: Both policies utilize frequency as a core metric. Mathematically, LFU can be viewed as a special case of Time-Decayed Caching where the decay rate approaches zero (infinite half-life), meaning past accesses never lose their weight.
- Differences: The critical flaw of standard LFU is its inability to handle "cache pollution" caused by non-stationary workloads. An item that was popular in the past but is no longer relevant may retain a high count and block new, currently popular items. TDC solves this by introducing the time-decay factor which ensures that old popularity mass naturally fades over time.
- Expected Performance: We anticipate similar results to LFU on purely stationary workloads where the popular items never change. However, on non-stationary workloads (where the popularity distribution shifts periodically), we expect TDC to yield significantly better hit rates. Our policy is designed specifically to "forget" outdated history, allowing it to detect and adapt to new hot sets much faster than LFU.

Adaptive Replacement Cache (ARC)

ARC is a sophisticated, self-tuning algorithm that dynamically balances between recency (LRU) and frequency (LFU). It maintains two lists: one for items accessed once (recency) and another for items accessed at least twice (frequency), along with "ghost" lists that track evicted items to adjust the cache partition size adaptively.

Comparison with Proposed Policy

- Similarities: Both ARC and TDC aim to solve the recency-frequency trade-off. Both are considered "scan-resistant" and high-performance policies.
- Differences: ARC achieves this balance structurally (by resizing lists), whereas TDC achieves it mathematically (through the half-life decay formula). TDC offers a simpler implementation model, requiring only a priority queue (min-heap) and a single scalar score per item, avoiding the complexity of maintaining four distinct lists and shadow directories.
- Expected Performance: We consider ARC the strongest baseline. We expect TDC to achieve competitive hit rates comparable to ARC across most traces.

Other Replacement Policies

TinyLFU [6] and LIRS [7] are also relevant state-of-the-art or near state-of-the-art policies; however, we do not include them in our comparisons and experiments.

Summary of Expectations

To summarize our comparative expectations, the proposed Time-Decayed Caching policy is designed to improve on existing literature (particularly LFU) in non-stationary environments. By mathematically coupling recency and frequency, TDC retains the high hit rates of frequency-based policies while matching the adaptability of recency-based ones.

Additionally, we expect TDC to surpass LRU in hit rate performance across most workloads because LRU ignores frequency entirely, whereas TDC's score accumulation preserves valuable, frequently accessed content. Against LFU, we anticipate a significant advantage on non-stationary workloads. Finally, compared to ARC, we expect TDC to remain competitive and comparable. While ARC achieves a similar recency-frequency balance through structural complexity, TDC offers a simpler implementation model while targeting the same high-performance metrics.

3. The Proposed Policy: Time-Decayed Caching

3.1. Formulation

We formalize the time-decayed caching policy in terms of a per-key decayed popularity score and an associated eviction rule. The key idea is that each access to an object contributes a unit of "popularity mass" that decays exponentially over subsequent requests, parameterized by a half-life H.

Problem setting and notation

We work in discrete time, indexed by request number. Let:

- $\mathcal{U}$ be the universe of objects.
- $r_1, r_2, \ldots, r_t, \ldots$ be the request sequence, where each $r_t \in \mathcal{U}$.
- The cache have capacity C (considered in number of objects).
- $\mathcal{C}(t) \subseteq \mathcal{U}$ denote the set of objects in the cache immediately before serving request r_t.

For each object $i \in \mathcal{U}$, let

$$t_1^i < t_2^i < \cdots < t_{k_i(t)}^i \leq t$$

be the times (request indices) at which i has been requested up to and including time t, and let $k_i(t)$ be the number of such requests.

Time-decayed popularity score

The policy associates to each object i a real-valued popularity score $S_i(t)$ at time t. This score is defined as an exponentially decayed sum of past accesses:

$$S_i(t) \;=\; \sum_{j=1}^{k_i(t)} e^{-\lambda(t-t_j^i)}, \lambda > 0.$$

Equivalently, if we parameterize the decay in terms of a **half-life** $H > 0$ measured in number of requests, we set

$$\lambda \;=\; \frac{\ln 2}{H}$$

and obtain the equivalent form

$$S_i(t) \;=\; \sum_{j=1}^{k_i(t)} 2^{-(t-t_j^i)/H}.$$

In this formulation:

- Each request to object i at time t_j^i contributes a term that is initially 1 and then decays over subsequent requests.
- After exactly H requests have passed since that access, its contribution is halved:

$$2^{-H/H} = \frac{1}{2}.$$

- After $2H$ requests, the contribution is $2^{-2} = \frac{1}{4}$, after $3H$ it is $2^{-3} = \frac{1}{8}$, and so on.

Thus, H defines an effective horizon over which past hits remain significantly influential: a hit that occurred nH requests ago counts as 2^{-n} of a new hit. Small values of H emphasize very recent activity, while large values integrate information over a longer history.

Conceptually, this popularity score has the following properties:

- It increases whenever the object is requested (each new hit contributes an additional term of size 1).

- It decreases smoothly as time passes without accesses, since every term in the sum is multiplied by a factor smaller than 1 at each step.
- It captures both frequency (more hits add more terms) and recency (more recent hits are less decayed and thus have larger weight).

For later implementation and analysis, it is helpful to note that this definition admits a recursive characterization. Let $\Delta_i(t)$ denote the number of requests that have occurred since the previous request for object i (i.e., the gap in request indices). Then, immediately before processing a new request for i at time t, we can write

$$S_i(t) \;=\; S_i(t^-) \cdot 2^{-\Delta_i(t)/H} + 1,$$

where $S_i(t^-)$ denotes the score right before applying the current hit. This shows that the full history of accesses to i can be compressed into a single scalar S_i, updated by exponential decay plus a unit increment on each hit.

Eviction rule

The time-decayed policy uses the scores $S_i(t)$ to make eviction decisions. At a cache miss at time t, when the cache is already full (i.e., $\mid \mathcal{C}(t) \mid = C$), the policy proceeds conceptually as follows:

1. For each object $j \in \mathcal{C}(t)$, consider its current time-decayed popularity score $S_j(t)$.
2. Identify an object with minimal score:

$$j = \arg\min_{j \in \mathcal{C}(t)} S_j(t).$$

3. Evict j from the cache.
4. Insert the newly requested object r_t into the cache and initialize its score with 1.

The half-life parameter H controls how rapidly the cache "forgets" past popularity:

- For small H, scores concentrate on recent accesses, and the policy behaves more like a recency-driven strategy.
- For large H, scores retain the influence of past accesses over longer spans, and the policy behaves more like a smoothed frequency-based strategy.

In subsequent sections we describe how to maintain the popularity scores efficiently in an online setting and how to adapt the half-life H to the observed workload.

3.2. Complexity

A naïve implementation would maintain, for each object i in the cache, its explicit decayed score

$$S_i(t) = \sum_{j=1}^{k_i(t)} e^{-\lambda(t - t_j^i)}$$

and, at a miss, recompute all $S_j(t)$ for $j \in \mathcal{C}(t)$ in order to select the victim with minimal score. This leads to $\mathcal{O}(\mid \mathcal{C}(t) \mid)$ work per miss, which is unacceptable for large caches. Instead, we exploit the algebraic structure of $S_i(t)$ to maintain a time-normalized key K_i for each cached object and store these keys in a priority queue (min-heap), so that extracting the victim remains efficient.

For an object i, let t_i denote the time of its most recent update (hit) and $S_i(t_i)$ the score immediately after that update. By definition of exponential decay,

$$S_i(t) = S_i(t_i) e^{-\lambda(t - t_i)}$$

for any later time $t \geq t_i$. We can rewrite this as

$$S_i(t) = e^{-\lambda t}(S_i(t_i)\, e^{\lambda t_i}).$$

This suggests defining the time-normalized key

$$K_i := S_i(t_i)e^{\lambda t_i}.$$

Then, for any current time t,

$$S_i(t) = e^{-\lambda t}K_i.$$

The global factor $e^{-\lambda t}$ is the same for all objects, so for any fixed t the ordering of objects by true score $S_i(t)$ is exactly the same as the ordering by key K_i. In particular, an object with minimal K_i also has minimal current score $S_i(t)$. This allows us to store only K_i for each object and to use K_i as the heap key, without explicitly recomputing all $S_i(t)$ at eviction time.

Moreover, the key representation yields a simple update rule on hits that does not require storing $t_i$or $S_i(t_i)$ separately. Suppose object i is present in the cache at time t, with current key K_i. Its true score just before processing the new hit is

$$S_i^{\text{current}} = S_i(t) = K_i e^{-\lambda t}.$$

After the hit, the new score is

$$S_i^{\text{new}} = S_i^{\text{current}} + 1 = K_i e^{-\lambda t} + 1.$$

The corresponding new key is

$$K_i^{\text{new}} = S_i^{\text{new}} e^{\lambda t} = \left(K_i e^{-\lambda t} + 1\right) e^{\lambda t} = K_i + e^{\lambda t}.$$

Thus, on a hit at time t, we can update the key using the simple recurrence

$$K_i \leftarrow K_i + e^{\lambda t},$$

without explicitly tracking $t_i$or $S_i(t_i)$. On a miss for a never-seen-before object i, its first request occurs at time t, and by the definition of $S_i(t)$ we have $S_i(t) = 1$, so we initialize

$$K_i = e^{\lambda t}.$$

To support efficient eviction, we maintain a hash table from object identifiers to heap positions, and a binary min-heap keyed by K_i over the objects currently in the cache. For each request, the operations are:

- Hit: we look up the object in the hash table (expected $\mathcal{O}(1)$), compute $e^{\lambda t}$ once for the current time step, update $K_i \leftarrow K_i + e^{\lambda t}$, and perform a key-increase operation in the heap. In a binary heap this takes $\mathcal{O}(logC)$ time, where C is the cache capacity.
- Miss with free space: we insert a new object with key $K_i = e^{\lambda t}$ into the hash table and heap, which costs $\mathcal{O}(logC)$.
- Miss with full cache: we extract the minimum key from the heap to obtain the victim

$$j = arg\,\underset{j \in \mathcal{C}(t)}{min}\, K_j$$

in $\mathcal{O}(logC)$, remove j from the hash table, and then insert the new object as above, for a total $\mathcal{O}(logC)$ time.

Therefore, each request is processed in $\mathcal{O}(logC)$ worst-case time, dominated by heap operations, compared to the $\mathcal{O}(1)$ updates of pointer-based LRU or counter-based LFU. In practice, cache capacities are bounded and $logC$ is small (e.g., $log_2 10^4 \approx 14$), so the additional overhead is modest, while the policy gains the ability to track a smooth time-decayed popularity measure without scanning the full cache. The space complexity is $\mathcal{O}(C)$: for each cached object we store its key K_i, the cached value (or a pointer to it), and the metadata required by the hash table and heap.

4. Implementation

This chapter aims to describe how the time-decayed caching policy is implemented in practice. We first outline the data structures and state maintained by the cache, and then present the core operations in pseudocode. A full C++ implementation is available as open-source code on GitHub [8].

4.1. Data Structures and Maintained State

The implementation follows the formulation from section 3.2. Thus, we store only a single scalar key K_i per cached object, rather than explicitly maintaining its decayed score $S_i(t)$. We recall that for an object i updated last at time t_i with score $S_i(t_i)$, we define

$$K_i \ := S_i(t_i)e^{\lambda t_i},$$

so that for any current time t,

$$S_i(t) = e^{-\lambda t}K_i.$$

Because the factor $e^{-\lambda t}$ is the same for all objects, the ordering of objects by K_i coincides with the ordering by their true decayed scores $S_i(t)$ at time t.

The cache maintains the following global state:

- A global time index $t \in \mathbb{N}$, interpreted as the number of read requests processed so far. Each access operation increments t by one.
- The cache capacity C, measured in number of objects.
- The decay parameter $\lambda > 0$, computed based on half-life H: $\lambda = ln2/H$.

For the objects currently stored in the cache, we maintain two core data structures:

- A hash table hashTable that maps object identifiers to internal cache entries. This provides expected $\mathcal{O}(1)$ lookup by key.
- A binary min-heap minHeap containing all cached entries, ordered by their time-normalized key K_i. The heap supports extracting the entry with minimal K_i and updating an existing entry in $\mathcal{O}(\log C)$ time.

Each cached object i is represented by a cache entry that stores:

- object: the identifier of the cached object,
- K: the current time-normalized key K_i associated with that object.

The explicit score $S_i(t)$ and the last update time t_i are not stored. Conceptually, they can be recovered from K_i, but the reconstruction is not required by the core operations of the policy.

4.2. Core Operations

We model cache usage through a single access operation on objects. Each request in a trace corresponds to one call to ACCESS(object), which either hits or misses in the cache. The cache tracks only which objects are present and their time-decayed popularity scores.

On a hit, the cached entry's time-normalized key K_i is updated. On a miss, if the object has never been seen before, its first request at time t should give a decayed score $S_i(t) = 1$. By the definition, this corresponds to initializing

$$K_i = e^{\lambda t}.$$

If the cache is full, the least popular entry is evicted before inserting the new object.

In the pseudocode below, $\exp(\cdot)$ denotes the natural exponential $e^{(\cdot)}$. HEAP_UPDATE, HEAP_INSERT, and HEAP_POP_MIN represent classic heap operations on minHeap. Similarly, HASH_TABLE_SEARCH, HASH_TABLE_INSERT, and HASH_TABLE_REMOVE represent hash table operations on hashTable.

```
procedure ACCESS(object):
    t ← t + 1 // advance time counter
    entry ← HASH_TABLE_SEARCH(hashTable, object)

    if entry != NONE then // cache hit
      UPDATE_POPULARITY(entry)
      return HIT

    else // cache miss
      if SIZE(minHeap) = C then
         EVICT()

    entry ← new cache entry
    entry.object ← object
    entry.K ← exp(delta · t)

    HEAP_INSERT(minHeap, entry)
    HASH_TABLE_INSERT(hashTable, object, entry)

    return MISS
```

On a hit, we update the time-normalized key K_i for the corresponding entry. If the current time is t and the entry has key K_i, then after accounting for the decayed past and adding one new access, the updated key is

$$K_i \leftarrow K_i + e^{\lambda t}.$$

```
procedure UPDATE_POPULARITY(entry):
    delta ← exp( · t)
    entry.K ← entry.K + delta
    HEAP_UPDATE(minHeap, entry)
```

Here HEAP_UPDATE(minHeap, entry) performs the appropriate sift-up or sift-down operation to restore the min-heap invariant after entry.K changes; in a binary heap this runs in $O(\log C)$time.

```
procedure EVICT():
    victim ← HEAP_POP_MIN(minHeap)
    HASH_TABLE_REMOVE(hashTable, victim.object)
```

During eviction, the cache entry with the smallest time-normalized key K_i, i.e. the least popular object according to the time-decayed score, is removed from minHeap. Its mapping from hashTable is removed as well.

5. Evaluation

5.1. Methodology

We aim to describe how we evaluate the proposed time-decayed cache replacement policy ("Proposed") against three baselines: LRU [2], LFU [3] and ARC [5]. We first define the validation metrics, then describe the experimental design, and finally present the workloads and the choice of decay parameters.

Validation metrics and baselines

We compare four algorithms:

$$\mathcal{A} = \{LRU, LFU, ARC, Proposed\}.$$

A request trace is a finite sequence

$$R = (r_1, r_2, \ldots, r_T),$$

where r_t is the identifier of the object requested at time t, and T is the trace length.

For each algorithm $A \in \mathcal{A}$, cache capacity C, and trace R, we simulate the trace and record, for every request $t \in \{1, \ldots, T\}$, a hit indicator

$$H_t(A, C, R) \in \{0, 1\},$$

which is 1 if the request r_t is a cache hit under algorithm A with capacity C, and 0 otherwise.

The hit ratio and miss ratio are defined as

$$HR(A, C, R) = \frac{1}{T} \sum_{t=1}^{T} H_t(A, C, R), MR(A, C, R) = 1 - HR(A, C, R).$$

The hit ratio HR is the main effectiveness metric.

To normalize performance across heterogeneous traces and cache capacities, we use a relative gap-to-best measure. Let traces be indexed by j, and capacities by an index c corresponding to a capacity level (defined precisely further in this section). For a fixed trace R_j and capacity level c, we define the best hit ratio among all algorithms as

$$HR_{\max}(j, c) = \max_{A' \in \mathcal{A}} HR(A', C_{j,c}, R_j),$$

where $C_{j,c}$ is the absolute cache capacity (number of objects) corresponding to level c on trace j.

For the proposed algorithm we define, on each trace j and capacity level c, the relative gap

$$\Delta(j, c, Proposed) = \frac{HR(Proposed, C_{j,c}, R_j) - HR_{\max}(j, c)}{HR_{\max}(j, c)} \cdot 100\%.$$

A value $\Delta(j, c, Proposed) > 0\%$ means that the proposed policy is better than another policy.

To study adaptivity on non-stationary traces (i.e., traces with periodically changing popular items), we additionally use a sliding-window hit ratio. We fix a window length W. For $t \geq W$, the windowed hit ratio is

$$HR_W(A, C, R, t) = \frac{1}{W} \sum_{\tau=t-W+1}^{t} H_\tau(A, C, R).$$

On non-stationary traces, we plot $HR_W(A, C, R, t)$ over time for all algorithms $A \in \mathcal{A}$ and visually compare:

- the drop in hit ratio after each phase change, and
- the speed at which each algorithm's HR_W recovers.

This highlights how quickly the proposed policy adapts relative to LRU, LFU and ARC.

Experimental design

We assume a finite object universe

$$\mathcal{O} = \{o_1, o_2, \ldots, o_N\},$$

where each o_i is a distinct object identifier. Each trace $R = (r_1, \ldots, r_T)$ is a sequence of requests with $r_t \in \mathcal{O}$.

For a given trace R and length T, we define the footprint

$$U(R, T) = \mid \{r_t : 1 \leq t \leq T\} \mid,$$

the number of distinct objects requested in the first T requests. In our experiments, T is the full trace length; we write $U_j = U(R_j, T_j)$ for trace j with length T_j.

Cache capacities are expressed as percentages of the footprint. We fix a set of capacity fractions

$$\alpha_c \in \{0.01, 0.02, 0.04, 0.08, 0.16\},$$

corresponding to 1%, 2%, 4%, 8% and 16% of the footprint. For trace j and capacity level c, the absolute cache capacity (in number of objects) is

$$C_{j,c} = \lfloor \alpha_c \cdot U_j \rfloor.$$

All algorithms are evaluated on the same pairs $(R_j, C_{j,c})$.

We consider two main experiments:

1. **Overall performance experiment**:

For each trace $R_j$and each capacity level α_c:

a. Compute U_j and the corresponding capacity

$$C_{j,c} = \lfloor \alpha_c \cdot U_j \rfloor.$$

b. For each algorithm $A \in \mathcal{A}$, simulate the full trace R_j with capacity $C_{j,c}$, record the hit indicators $H_t(A, C_{j,c}, R_j)$ and compute the hit ratio $HR(A, C_{j,c}, R_j)$.
c. From the resulting set of hit ratios, compute $HR_{\max}(j, c)$ and the relative gap $\Delta(j, c, Proposed)$ for the proposed algorithm.
d. The statistics defined in the previous subsection are then computed over all values $\Delta(j, c, Proposed)$ obtained in this experiment.

2. **Adaptivity experiment:**

To isolate adaptivity, we consider non-stationary traces with a known number of phases P, each of equal length L, so that the total length is

$$T = P \cdot L.$$

Each phase $p \in \{1, \ldots, P\}$ has its own hot set and request distribution; phase changes occur at times

$$t = L, 2L, \ldots, (P-1)L.$$

a. Simulate the full non-stationary trace R with capacity C, recording the hit indicators $H_t(A, C, R)$.
b. For each $t \geq W$, compute the sliding-window hit ratio

$$HR_W(A, C, R, t) = \frac{1}{W} \sum_{\tau=t-W+1}^{t} H_\tau(A, C, R).$$

c. We then plot $HR_W(A, C, R, t)$ over time for all algorithms. Adaptivity is judged qualitatively by the magnitude of the drop in HR_W immediately after phase changes, and the recovery speed (i.e., how quickly HR_W returns to its steady level in each new phase).

Additionally, we also run this experiment for the real-world trace.

Workloads and decay parameter selection

We use four synthetic workloads and one real-world trace. Synthetic workloads let us control the popularity dynamics; the real trace demonstrates behavior under realistic access patterns. For all experiments, the baselines use standard parameter settings, and the proposed policy uses a decay parameter λ chosen according to the rules below.

Stationary synthetic workloads

In the stationary workloads, popularity does not change over time. In total, we will be using two workloads generated from a Zipf [3] distribution over $\mathcal{O}$. We index objects as $o_1, \ldots, o_N$, and define

$$\mathbb{P}(r_t = o_i) \propto i^{-\alpha},$$

for indices $i \in \{1, \ldots, N\}$, where the rank i corresponds to object o_i. More specifically, the object with rank 1 will be hottest, rank 2 is colder, and so on. This gives us a heavy-tailed popularity: a few objects get a lot of requests, and there is a long tail of rarely used ones. In order for the probabilities to sum up to 1 and maintain the proportionality described above, we use the following probability formula

$$\mathbb{P}(r_t = o_i) = \frac{i^{-\alpha}}{\sum_{j=1}^{N} j^{-\alpha}}.$$

We will use a different choice of α for each of the two workloads (e.g., moderately and highly skewed values).

Non-stationary synthetic workloads

In the non-stationary workloads, popularity changes over time in a controlled way. We will be using two workloads that consist of P phases of equal length L, so that $T = P \cdot L$. For each phase $p \in \{1, \ldots, P\}$, we choose a hot set $H_p \subset \mathcal{O}$ of size k, with limited overlap between consecutive hot sets H_p and H_{p+1}. Within phase p, requests are generated as

$$r_t = \begin{cases} \text{a key chosen uniformly from } H_p \text{ with probability } p_{hot} \\ \text{a key chosen uniformly from } \mathcal{O} \setminus H_p \text{ with probability } 1 - p_{hot} \end{cases}.$$

The hot set thus "jumps" at times $t = L, 2L, \ldots$, creating clear phase changes.

Real-world trace

In addition to the synthetic workloads, we evaluate the algorithms on a real-world trace drawn from a web server cache. This serves as a realistic counterpart to the previous non-stationary workloads, since web-access traces are known to exhibit strong temporal locality and evolving hot sets as user interests shift over time.

We will use the NASA web server trace from July 1995 (Jul95 [9]), a widely used benchmark consisting of HTTP requests to a public NASA site. This trace is simple to parse (one request per line) yet exhibits realistic web access patterns, making it suitable for evaluating cache replacement policies.

Decay parameter selection

The proposed policy uses exponential decay with rate $\lambda > 0$, equivalently described by a half-life H:

$$H = \frac{\ln 2}{\lambda}.$$

We choose λ based on a desired memory horizon A (in units of requests), and then keep λ fixed across all traces within a given regime:

1. Stationary workloads

Let T be the number of simulated requests per stationary trace. We choose a memory horizon

$$A = \beta \cdot T,$$

$$H = \frac{A}{3}, \lambda = \frac{\ln 2}{H}.$$

With this choice, hits that occurred about A requests ago have their weight reduced by a factor of

$$2^{A/H} = 2^3 = 8,$$

2. Non-stationary workloads

Let L denote the characteristic length of a stable hot phase. We choose a memory horizon A on the order of L (for example $A = L$) and again set

$$H = \frac{A}{3}, \lambda = \frac{\ln 2}{H}.$$

3. Real-world trace

We select the decay parameter λ using the same procedure as for the generated stationary workloads.

5.2. Case Study: Synthetic Cache Traces

To evaluate the proposed Time-Decayed Caching (TDC) policy, we conducted experiments on four synthetic traces representing different access patterns. The traces were generated with 1,000 unique items and 50,000 requests each:

- Stationary Zipf (α = 0.8): A moderately skewed distribution where popular items receive more accesses, but the popularity is relatively spread out.
- Stationary Zipf (α = 1.2): A highly skewed distribution with strong concentration of accesses on a small set of popular items.
- Non-Stationary (p_hot = 0.7): A trace with 10 phases where the hot set changes between phases. In each phase, 70% of requests target the hot set (25% of items).
- Non-Stationary (p_hot = 0.9): Similar phase-based workload but with 90% of requests targeting the hot set, creating more pronounced locality shifts.

We compared the proposed TDC algorithm against three baseline policies: Least Recently Used (LRU), Least Frequently Used (LFU), and Adaptive Replacement Cache

(ARC). Experiments were conducted for cache sizes ranging from 1% to 16% of the total item space.

Experiment 1: Hit Ratio Performance

Table 1 presents the average Δ(j, c, Proposed) values across all cache sizes, measuring the relative difference between the proposed algorithm's hit ratio and each baseline. Positive values indicate that the proposed algorithm achieves a higher hit ratio than the baseline.

Average Δ(j, c, Proposed) Across All Cache Sizes (%)
(Positive = Proposed outperforms baseline)

Trace	Δ vs LRU	Δ vs LFU	Δ vs ARC
Stationary (α=0.8)	+16.84%	-11.71%	-8.42%
Stationary (α=1.2)	+5.20%	-4.39%	-2.14%
Non-Stationary (p_hot=0.7)	+11.20%	+33.56%	-5.17%
Non-Stationary (p_hot=0.9)	+7.97%	+39.56%	-1.96%

Table 1. Average $\Delta(j, c, \text{Proposed})$ across all cache sizes on the synthetic traces (positive means Proposed outperforms the baseline).

The results demonstrate TDC's performance characteristics across different workload types:

- **vs LRU**: TDC consistently outperforms LRU across all traces, with improvements ranging from +5.20% to +16.84% on stationary traces and +7.97% to +11.20% on non-stationary traces. This demonstrates that incorporating frequency information through time-decay provides significant benefits over pure recency-based eviction.
- **vs LFU**: On stationary traces, LFU outperforms TDC (Δ of -11.71% and -4.39%), which is expected since LFU is optimized for stable access patterns. However, on non-stationary traces, TDC significantly outperforms LFU with improvements of +33.56% and +39.56%. This highlights the critical weakness of LFU in dynamic environments where access patterns change over time.
- **vs ARC**: ARC, being an adaptive algorithm, performs competitively. TDC shows a very small gap to ARC on most traces (-8.42% to -1.96%), with the gap narrowing on non-stationary workloads where TDC's time-decay mechanism proves beneficial.

Experiment 2: Adaptability Analysis

To further investigate the adaptability of each algorithm, we analyzed the sliding window hit ratio over time on the non-stationary traces using a 5% cache size and a window of 500 requests.

Figure 1 reveals several important observations:

- LFU struggles with phase changes: The LFU algorithm shows significant performance drops after each phase transition, as its frequency counters retain stale information from previous phases.
- LRU adapts quickly but has lower peak performance: LRU recovers rapidly from phase changes due to its focus on recency, but its steady-state performance is limited.
- ARC demonstrates strong adaptive behavior: The ARC algorithm balances recency and frequency effectively, showing competitive performance across phases. This is expected, as ARC is near-SOTA on most cache traces.

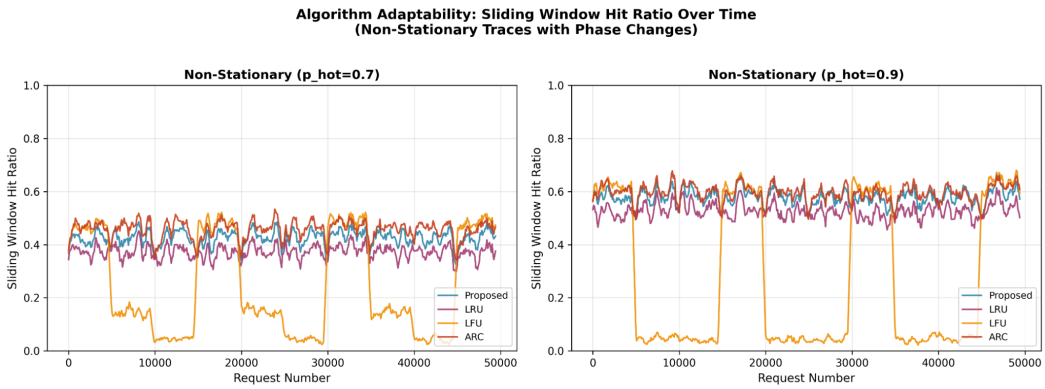

Figure 1. Sliding-window hit ratio over time on the non-stationary synthetic traces with phase changes (cache size: 5% of footprint; window: 500 requests).

- TDC achieves balanced adaptation: The proposed algorithm demonstrates smooth transitions between phases while maintaining competitive hit ratios. The time-decay mechanism naturally deprioritizes stale frequency information, enabling faster adaptation than pure LFU while preserving frequency benefits over pure LRU. The TDC performance was closest to ARC.

Discussion

The experimental results validate the design goals of the Time-Decayed Caching policy:

- Unified recency-frequency handling: By using exponential time decay, TDC captures both recency (recent accesses have higher scores) and frequency (repeated accesses accumulate score) in a single, elegant mechanism.
- Adaptation: The time-decay factor allows the algorithm to naturally "forget" outdated access patterns, enabling smooth adaptation to workload changes.
- Performance: While TDC may not achieve the absolute best performance on some workloads (e.g., purely stationary high-skew traces where LFU excels), it consistently performs within a small margin of the best algorithm across diverse workload types.

5.3. Case Study: Real-World Cache trace

To complement the synthetic workloads, we also evaluate the algorithms on a real web-access trace. We use the NASA HTTP server log [9] from July 1995 (Jul95), a widely used benchmark consisting of HTTP requests to a public NASA site. Each request line is mapped to an object identifier given by the requested URL path. As in the synthetic experiments, cache capacities range from 1% to 16% of the distinct object footprint, and we compare the proposed Time-Decayed Caching (TDC) policy against LRU, LFU, and ARC.

Experiment 1: Hit Ratio Performance

Table 2 reports the average $\Delta(j, c, \text{Proposed})$ values over all cache sizes for the NASA trace. Positive values indicate that the proposed policy achieves a higher hit ratio than the baseline, while negative values indicate that the baseline performs better on average.

On this real workload, TDC outperforms LRU by +1.79% on average, confirming that incorporating time-decayed frequency information gives a measurable advantage over a purely recency-based rule. LFU and ARC achieve slightly higher hit ratios than TDC, with average gaps of -3.86% and -3.62%, respectively. These differences are modest, showing that the proposed policy remains competitive with strong, frequency-aware baselines even on a real trace where no explicit synthetic phase structure is imposed.

Experiment 2: Adaptability Analysis

Average Δ(j, c, Proposed) Across All Cache Sizes (%)
(Positive = Proposed outperforms baseline)

Trace	Δ vs LRU	Δ vs LFU	Δ vs ARC
NASA HTTP Log	+1.79%	-3.86%	-3.62%

Table 2. Average $\Delta(j, c, \text{Proposed})$ across all cache sizes on the NASA HTTP trace (positive means Proposed outperforms the baseline).

To study short-term behavior, we compute the sliding-window hit ratio on the NASA log using the same configuration as in the synthetic adaptivity experiment: a cache size of 5% of the footprint and a window of 500 requests. Figure 2 plots the resulting curves for all four algorithms over the first 100,000 requests.

All algorithms exhibit consistently high hit ratios (roughly between 0.7 and 0.85), with only moderate fluctuations over time. The curves for TDC, LRU, LFU, and ARC closely track one another, and no algorithm experiences the sharp collapses seen for LFU on the synthetic non-stationary traces. This reflects the more gradual, organic evolution of popularity in the real log: while the hot set does change over time, it does so without abrupt, engineered phase boundaries. Within this environment, TDC remains stable and closely follows ARC and LFU, with a small but persistent lead over LRU.

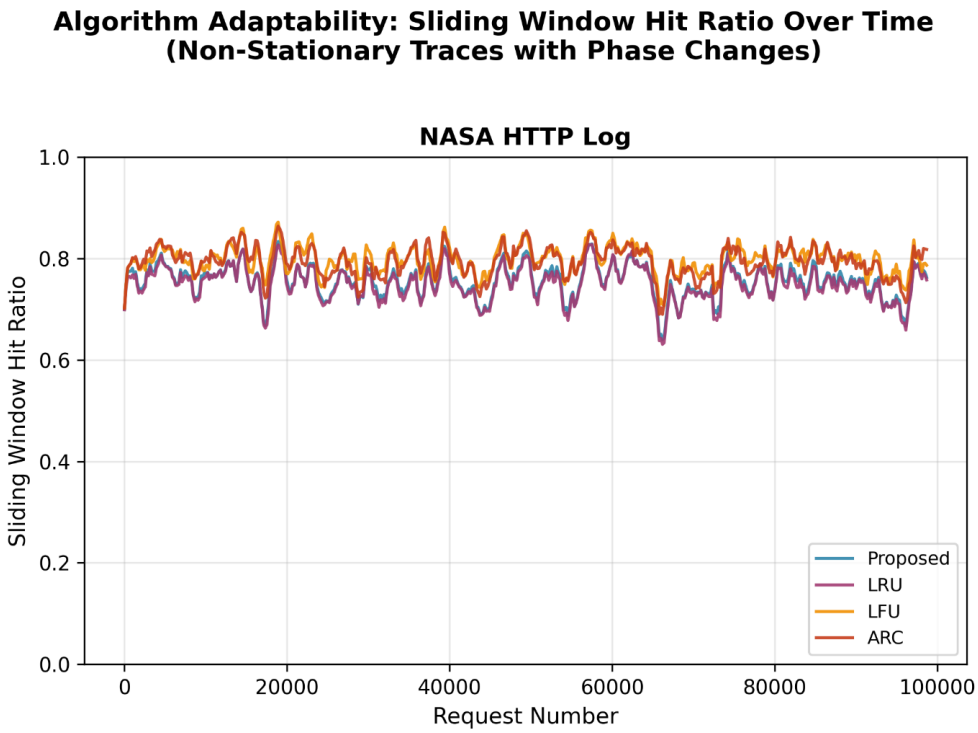

Figure 2. Sliding-window hit ratio over time on the NASA HTTP log (first 100,000 requests; cache size: 5% of footprint; window: 500 requests).

Discussion

The NASA case study shows that the proposed time-decayed policy behaves robustly on a real-world workload. It delivers a consistent improvement over LRU and stays within a few percentage points of LFU and ARC across cache sizes, while maintaining smooth, high sliding-window hit ratios over time. These results indicate that the mathematical time-decay mechanism transfers effectively to realistic web-access patterns.

6. Conclusion

This work proposed Time-Decayed Caching (TDC), a replacement policy that integrates recency and frequency through an exponential half-life model of popularity. Each object maintains a decayed score that aggregates past hits with a tunable half-life parameter H, providing an explicit notion of how long historical evidence should matter. By introducing time-normalized keys and storing them in a binary min-heap, we obtained an implementation that answers each request in $O(\log C)$ worst-case time for cache capacity C, while avoiding global score updates or additional auxiliary lists. The resulting design is conceptually simple, mathematically interpretable, and practical to implement in standard programming environments.

The evaluation addressed the research questions formulated in the introduction. On stationary Zipf workloads, TDC consistently improved upon LRU and stayed reasonably close to LFU and ARC, confirming that time-decayed scores can match the effectiveness of classic frequency-aware policies when the hot set is stable. On non-stationary workloads with phase changes, TDC clearly outperformed LFU, while maintaining a small gap to ARC. The sliding-window analysis showed that TDC quickly recovered its hit ratio after each shift in the hot set, combining much of LRU's adaptability with the long-term benefits of frequency. Finally, on the NASA HTTP trace, TDC delivered a measurable improvement over LRU and remained within a few percentage points of LFU and ARC across cache sizes, demonstrating that the time-decay mechanism transfers robustly to realistic web-access patterns.

Taken together, these results indicate that TDC is a viable alternative to more complex adaptive policies: it offers a clear, one-parameter control over the cache's memory horizon, achieves competitive hit ratios on both synthetic and real traces, and does so with modest computational overhead. Future work can build on this foundation by exploring adaptive or learned choices of the half-life parameter, extending the formulation to size-aware or cost-aware settings, and integrating TDC into production caching systems to study its behavior under real deployment constraints.

References

1. OpenStax. *College Physics*. OpenStax: Houston, TX, USA, 2012.
2. Silberschatz, A.; Galvin, P.B.; Gagne, G. *Operating System Concepts*, 9th ed.; John Wiley & Sons: Hoboken, NJ, USA, 2013.
3. Tanenbaum, A.S.; Bos, H. *Modern Operating Systems*, 4th ed.; Pearson: Upper Saddle River, NJ, USA, 2015.
4. Newman, M.E.J. Power laws, Pareto distributions and Zipf's law. *Contemporary Physics* **2005**, *46*, 323–351.
5. Megiddo, N.; Modha, D.S. ARC: A self-tuning, low overhead replacement cache. In *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST '03)*; San Francisco, CA, USA, 2003; pp. 115–130.
6. Einziger, G.; Friedman, R.; Manes, B. TinyLFU: A highly efficient cache admission policy. *arXiv* **2015**, arXiv:1512.00727.
7. Jiang, S.; Zhang, X. LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance. In *Proceedings of the 2002 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*; Marina del Rey, CA, USA, 2002; pp. 31–42.
8. Pângăleanu, C. Time-Decayed Caching: Reference Implementation. GitHub repository, 2025. Available online: https://github.com/PanCat26/time-decayed-caching.

9. Internet Traffic Archive. Two Months of HTTP Logs from the KSC-NASA WWW Server (NASA-HTTP). Lawrence Berkeley National Laboratory, 1995.

https://doi.org/10.3390/a1010000